

PARRY PAWS



CONTENTS

1. 기존 기획 내용
2. 주요 기능과 특징
3. 데모 플레이
4. 개발 과정
5. 향후 계획



기존 기획 내용

1. 공격보다 패링 중심의 전투 설계
2. 패링 난이도: 쉬운 타이밍 판정+빠른 템포
3. 성장 요소: 스킬 업그레이드로 보다 강해지는 주인공
4. 스페셜 시스템: 패링 성공 시 게이지 획득



주요 기능과 특징

1. 단순한 콤보 공격의 강력한 위력
2. 패링에 상태 이상 효과를 부여하여 패링 활용 유도
3. 구르기 기술로 공격 회피
4. 2단 점프로 챙기는 기동성

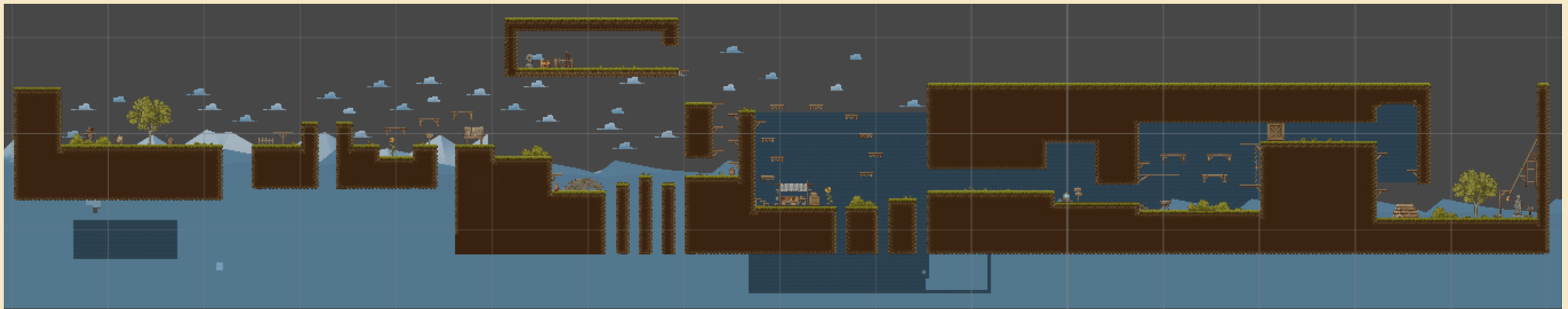




데모 플레이

개발 과정_맵

- 맵 구현(타일맵)



- 파라allax 배경 이동 기능

```
using UnityEngine;
namespace Platformer.View
{
    public class ParallaxLayer : MonoBehaviour
    {
        public Vector3 movementScale = Vector3.one;
        Transform _camera;

        void Awake()
        {
            _camera = Camera.main.transform;
        }

        void LateUpdate()
        {
            transform.position = Vector3.Scale(_camera.position, movementScale);
        }
    }
}
```

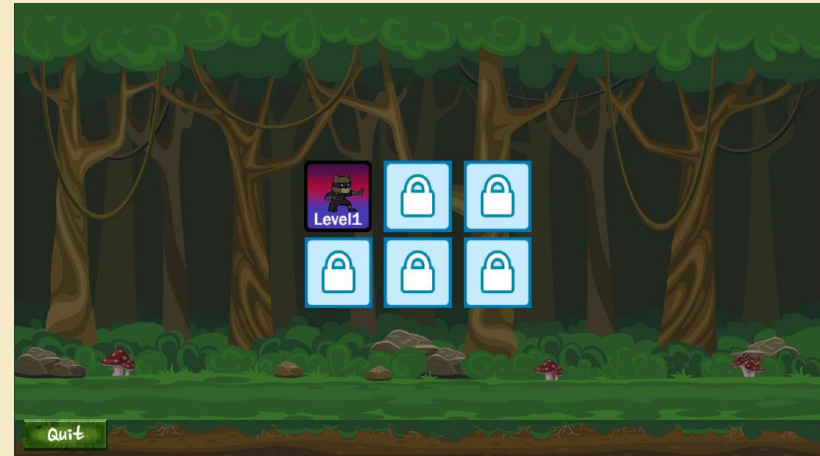


개발 과정_UI

- UI 구현(체력바)

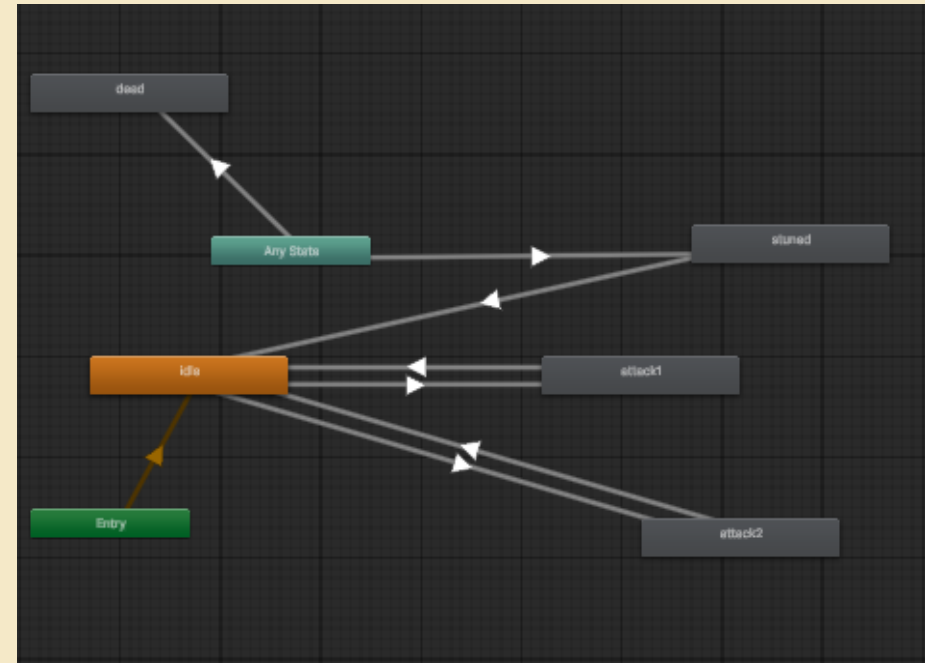
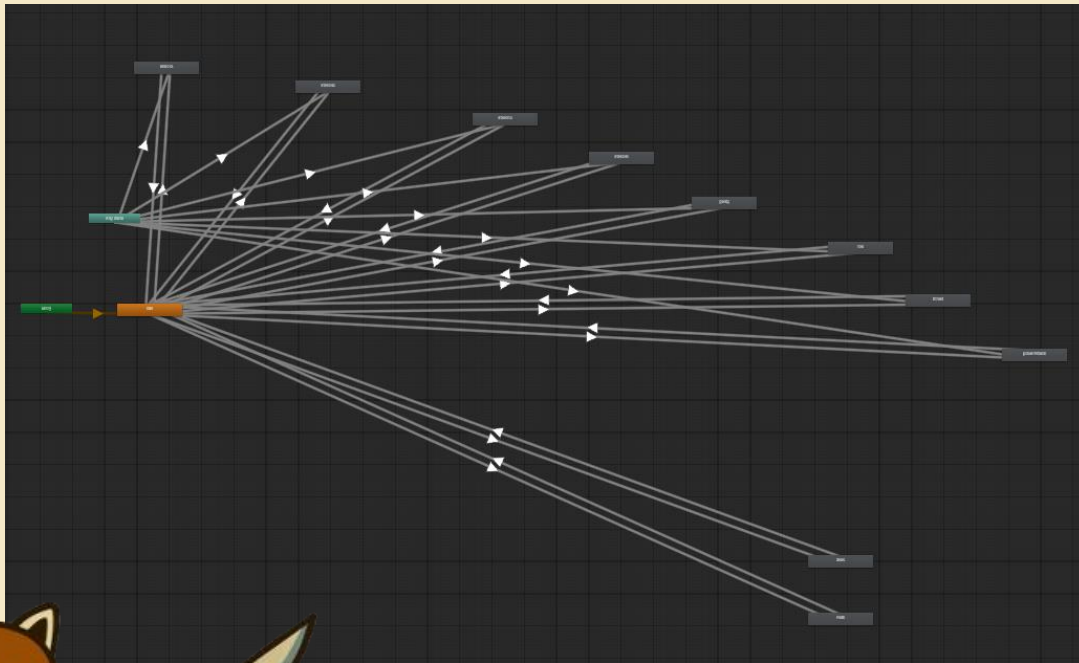


스테이지 선택 및 시작화면



개발 과정_캐릭터 및 로직

- 캐릭터 구현(애니메이션)



개발 과정_캐릭터 및 로직

- 캐릭터 구현(주인공)



- 공격 사거리 Collider

```
void AttackInput()
{
    if (anim == null) return;

    // 좌클릭 : 공보 공격
    if (Input.GetMouseButtonDown(0))
    {
        AnimatorStateInfo state = anim.GetCurrentAnimatorStateInfo(0);

        if (state.IsName("idle") || state.IsName("walk"))
        {
            anim.SetTrigger("attack1");
        }
        else if (state.IsName("attack1"))
        {
            anim.SetTrigger("attack2");
        }
        else if (state.IsName("attack2"))
        {
            anim.SetTrigger("attack3");
        }
        else if (state.IsName("attack3"))
        {
            anim.SetTrigger("attack4");
        }

        // 입력 시점에 바로 공격 판정
        DoMeleeHit();
    }
}
```

- 기본 공격 콤보 설계

```
private void OnTriggerEnter2D(Collider2D other)
{
    // 1) 이미 반사된 화살일 때 : 개마귀 (본체 또는 히트박스)에 맞았을 경우
    if (isReflected)
    {
        // 개마귀 컨트롤러는 Crow 본체에 있고, 히트박스는 자식에 있을 수 있으므로 InParent 사용
        CrowWitcherController crow = other.GetComponentInParent<CrowWitcherController>();
        if (crow != null)
        {
            // 개마귀가 한 번 맞았다고 알릴 (내부에서 맞은 횟수/HP만 처리)
            crow.OnHit();

            // 아직 죽지 않았다면 스펀 처리
            if (!crow.IsDead)
            {
                crow.Stun(crow.stunDuration);
            }

            // 화살 파괴
            Destroy(gameObject);
            return;
        }
    }

    // 2) 아직 반사되지 않은 화살 → 플레이어 피격 / 패링 처리
    if (!isReflected)
    {
        if (other.CompareTag("Player"))
        {
            // 플레이어가 자식에 있고 PlayerController2D가 부모에 있을 수도 있으므로 InParent 사용
            PlayerController2D player = other.GetComponentInParent<PlayerController2D>();
            if (player != null)
            {
                if (player.IsParrying)
                {
                    // 패링 상태이면 화살을 반사
                    ReflectBackToOwner();
                    return;
                }
                else
                {
                    // 패링이 아니면 플레이어가에게 데미지 전달
                    player.HitByProjectile(damageToPlayer);
                    Destroy(gameObject);
                    return;
                }
            }
        }
    }
}
```

- 패링 기능 설계



개발 과정_캐릭터 및 로직

- 캐릭터 구현(적)

```
private void ReflectBackToOwner()
{
    isReflected = true;

    if (owner == null)
    {
        Destroy(gameObject);
        return;
    }

    // 반사된 뒤에는 직선으로 날아가도록 중력 제거
    rb.gravityScale = 0f;

    // 현재 위치에서 원래 발사자(카마귀) 방향으로 다시 날려보냄
    Vector2 toOwner = ((Vector2)owner.transform.position - (Vector2)transform.position).normalized;
    float speed = rb.linearVelocity.magnitude;
    if (speed < 0.1f) speed = 8f; // 너무 느리면 기본 속도 보정

    rb.linearVelocity = toOwner * speed;

    // 방향에 따라 화살 스프라이트 좌우 반전
    float sign = toOwner.x >= 0 ? 1f : -1f;
    transform.localScale = new Vector3(
        sign * Mathf.Abs(baseScale.x),
        baseScale.y,
        baseScale.z
    );
}
```

- 패링된 투사체 처리

```
private IEnumerator ShootAttack1Routine()
{
    yield return new WaitForSeconds(attack1FireDelay);

    if (isDead || isStunned || player == null)
    {
        isAttacking = false;
        yield break;
    }

    Vector2 toPlayer = (player.position - firePoint.position);
    Vector2 dirToPlayer = toPlayer.normalized;

    GameObject arrowGo = Instantiate(arrowPrefab, firePoint.position, Quaternion.identity);
    ArrowProjectile arrow = arrowGo.GetComponent<ArrowProjectile>();
    if (arrow != null)
    {
        arrow.InitStraight(dirToPlayer, arrowSpeedStraight, gameObject);
    }

    useAttackNext = useAttackNext;
    isAttacking = false;
}

참조 1개
private IEnumerator ShootAttack2Routine()
{
    yield return new WaitForSeconds(attack2FireDelay);

    if (isDead || isStunned || player == null)
    {
        isAttacking = false;
        yield break;
    }

    Vector2 toPlayer = (player.position - firePoint.position);
    float horizSign = Mathf.Sign(toPlayer.x);
    if (Mathf.Abs(horizSign) < 0.01f)
        horizSign = isFacingRight ? 1f : -1f;

    GameObject arrowGo = Instantiate(arrowPrefab, firePoint.position, Quaternion.identity);
    ArrowProjectile arrow = arrowGo.GetComponent<ArrowProjectile>();
    if (arrow != null)
    {
        arrow.InitArc(horizSign, arrowSpeedArc, arcUpVelocity, gameObject);
    }

    useAttackNext = useAttackNext;
    isAttacking = false;
}
```

- 포물선, 직선 공격 변환 로직



향후 계획

1. 스테이지 다중화

2. 몬스터 종류 추가

3. 기존 레벨 선택 대신 작은 마을 형태의 거점 맵 추가

4. 시네머신 추가

